

AIC 2026 – Phân tích cuộc thi & Brainstorm chiến lược toàn diện

Ghi chú phương pháp: Tài liệu này tổng hợp phân tích cuộc thi HCMC AI Challenge 2026 (truy vấn video-hình ảnh từ văn bản), brainstorm các hướng giải cho 4 dạng câu hỏi (T-KIS, QA, V-KIS, TRAKE) và đề xuất chiến lược tổng thể. Pipeline cụ thể được tách riêng tại file `AIC26_Pipeline.md`. Các phần đánh dấu `[cần verify]` là những thông tin về phiên bản model hoặc benchmark mà nhóm nên kiểm tra lại trước khi đưa vào sản xuất.

1. Tóm tắt điều hành

Bản chất bài toán: HCMC AI Challenge 2026 là một biến thể mở rộng của bài toán **Known-Item Search (KIS)** trên kho video tin tức tiếng Việt, kết hợp 4 dạng truy vấn không đồng nhất: T-KIS (truy vấn văn bản dân dân), QA (truy vấn + câu hỏi suy luận), V-KIS (truy vấn bằng đoạn video bịt âm thanh) và TRAKE (truy vấn chuỗi sự kiện theo thời gian).

Luận điểm chiến lược cốt lõi:

- Hàm điểm** $\text{Score} = \max(0, 500 - 100 \cdot \text{penalty} + f(\text{time}))$ có đạo hàm theo penalty lớn hơn rất nhiều so với theo time (mỗi lần sai trừ 100, trong khi $f(\text{time})$ chỉ trải đều trên $[0; 500]$ suốt toàn bộ thời gian truy vấn). Hệ quả: **chiến lược dominant là tối thiểu hóa số lần submit sai**, không phải tối đa hóa tốc độ. Đây là sự đảo ngược trực giác so với các giải coding.
- Bottleneck là UI và workflow truy vấn, không phải sức mạnh mô hình.** Mô hình SOTA (Perception Encoder, SigLIP 2, EVA-CLIP) đã đủ recall@100. Khoảng cách giữa các team đến từ tốc độ điều hướng kết quả, chất lượng metadata làm giàu offline, và khả năng phục hồi khi mô hình bị "đánh trúng vùng mù".
- Đa dạng hóa kênh truy hồi quan trọng hơn tinh chỉnh một kênh.** Vector search bằng PE/SigLIP mạnh ở semantic chung; OCR cứu các câu có chữ trên màn hình; ASR cứu các câu nghe nội dung; caption VLM cứu V-KIS; object/scene tag cứu các câu compositional. Không kênh nào thay thế được kênh khác.

Đề xuất tổng quát:

- Indexing offline đa kênh, dư thừa có chủ đích:** PE-G (hoặc EVA-CLIP/SigLIP 2) cho vector chính + VLM-generated structured captions + Whisper ASR + multilingual OCR + open-vocabulary object detection + scene/action tagging.
- Online retrieval lai (hybrid):** Vector search + BM25 trên text-fields + filter cứng theo điều kiện thuộc tính (thời tiết, góc máy, OCR string).
- Reranking 2 tầng:** RRF/weighted fusion → VLM cross-encoder rerank top-K nhỏ.
- UI tối ưu cho người vận hành** với phím tắt, voice-input, group-by-video, hiển thị Chain-of-Thought cho QA, và queue submit có xác nhận.

2. Phân tích bối cảnh cuộc thi

2.1. Đặc điểm dữ liệu BTC

Thành phần	Mô tả	Hệ quả thiết kế
Videos/	Video gốc	Cần re-extract keyframe dày hơn nếu cần frame mịn cho TRAKE; cần audio cho ASR
Keyframes/	Tổ chức theo thư mục Lxx_Vxxx/, file .jpg đánh số tăng dần	Tên file là khóa định danh chính khi submit; cần map frame ↔ timestamp qua metadata
Objects/ (Faster R-CNN, OpenImagesV4)	JSON object detection sẵn	Chất lượng giới hạn: OpenImagesV4 có ~600 lớp, không bao phủ tốt entities tin tức Việt Nam (logo đài, biển hiệu, đồng phục...). Nên chỉ dùng làm filter phụ, bổ sung bằng open-vocabulary detector
CLIP features (clip-ViT-B-32)	.npy thứ tự tăng dần	B-32 đã lỗi thời (~2021). Bắt buộc tự encode lại bằng backbone mạnh hơn
Metadata	JSON từ YouTube, có thể thiếu	Khai thác làm text-field search; xử lý fallback cho video thiếu metadata

Ba kết luận quan trọng từ đặc điểm dữ liệu:

- 1. Features và objects BTC chỉ là baseline tối thiểu. Coi chúng như sanity-check, không phải pillar chính của pipeline.
- 2. Mapping (keyframe_index → video_id, frame_in_video, timestamp) là chìa khóa. Mọi câu submit đều cần đúng định danh keyframe BTC. Solution: build một bảng tra (KeyframeRegistry) ngay từ đầu, chuẩn hóa toàn pipeline về định danh này.
- 3. Tự re-cut keyframe dày hơn (ví dụ 1 frame/giây hoặc dùng shot-boundary detector như TransNetV2) và gộp với keyframe BTC, loại trùng lặp bằng perceptual hashing (pHash) hoặc cosine similarity ≥ 0.97 trên feature mạnh. Tăng recall cho TRAKE và QA mà không phá tương thích submit (vì frame submit phải nằm trong keyframe BTC ⇒ luôn map ngược về frame BTC gần nhất).

2.2. Quy định tính điểm - phân tích định lượng

Score = max(0, 500 - 100 · penalty + f(time)), với f(time) ∈ [0; 500] nội suy tuyến tính theo thời gian còn lại.

Giả sử thời lượng câu là T giây, người chơi submit ở thời điểm t, ta có f(time) ≈ 500 · (T - t) / T. Tách trường hợp:

- **Đúng ngay, không penalty:** điểm $\approx \boxed{500 + 500 \cdot (T-t)/T}$. Tại $\boxed{t = 0.2T}$: ~900 điểm. Tại $\boxed{t = 0.8T}$: ~600 điểm. **Khoảng cách 300 điểm** giữa nhanh và chậm.
- **Đúng sau 1 sai:** điểm $\approx \boxed{400 + 500 \cdot (T-t)/T}$. Tại $\boxed{t = 0.4T}$: ~700 điểm. **Mất 200 điểm** so với đúng ngay tại cùng thời điểm.
- **Sai 5 lần (penalty=5):** $\boxed{-500 + 500 \cdot (T-t)/T \leq 0}$ cho mọi $\boxed{t}$. **Hết điểm.**

Hệ quả chiến lược:

- Đánh đổi: thà mất 30s suy nghĩ kỹ trước khi submit còn hơn submit nhanh và sai một lần. Vì $30s \times \boxed{500/T} \approx$ vài chục điểm, trong khi 1 sai = -100 cộng với thời gian phục hồi sau đó.
- Cần có "submit guard" trong UI: trước khi gửi, hiển thị xác nhận có CoT/đặc trưng nổi bật của frame được chọn để người vận hành kiểm tra lần cuối.
- **Riêng TRAKE:** đúng $\boxed{\geq n/2}$ sự kiện vẫn được 50% accuracy. Cần một **chế độ fallback**: nếu hết 70% thời gian mà chưa định danh hết, submit phần chắc chắn để ăn 50%.

2.3. Hình thức thi và bottleneck thực tế

Câu hỏi hiển thị ở màn hình trung tâm, người thi truy vấn trên máy cá nhân, submit qua DRES. Hai bottleneck dễ bị xem nhẹ:

1. **Nhập liệu thủ công chậm:** Người vận hành phải gõ lại query từ màn hình. Solution: voice-to-text (Whisper hoặc API) + LLM rewrite để chuẩn hóa thành query cấu trúc.
2. **Duyệt kết quả mỗi mắt:** Top-100 hiển thị dạng grid không sắp xếp $\Rightarrow$ phải scroll. Solution: group-by-video, đánh dấu cụm frame liên tục, tô màu theo điểm rerank, tooltip hiển thị caption/OCR/ASR snippet.

Luận điểm "UI > Method > Model" là chính xác vì:

- Model gain biên đã giảm: PE-G vs SigLIP 2 chênh ~1-2% recall@10 trên benchmark academic, nhưng chênh lệch này bị che mờ hoàn toàn bởi 10s tiết kiệm trên UI tốt.
- Method gain trung bình: hybrid fusion + rerank tốt có thể đẩy recall@10 thêm 5-10%, cứu được 2-3 câu khó/phiên.
- UI gain biên lớn: một UI tốt có thể giảm thời gian trung bình mỗi câu từ 2 phút xuống 1 phút $\Rightarrow$ điểm $\boxed{f(\text{time})}$ mỗi câu tăng ~100 $\Rightarrow$ nhân với 20 câu/phiên = **2000 điểm.**

3. Phân tích sâu từng dạng câu hỏi

3.1. Textual KIS (T-KIS)

Bản chất: Truy vấn văn bản tăng dần (mỗi 30s thêm gợi ý). Output $\boxed{\langle \text{video}, \text{frame} \rangle}$.

Quan sát chiến lược:

- Gợi ý đầu thường **chung chung** (ví dụ: "một đám cháy ở khu dân cư"). Gợi ý sau thường **đặc tả thuộc tính phân biệt** (ví dụ: "có biển hiệu màu đỏ", "phóng viên mặc áo xanh"). Vì vậy:

- **Không nên submit ở gợi ý 1** trừ khi rất chắc chắn (recall@1 rất cao, ví dụ top-1 cosine similarity vượt threshold rõ rệt so với top-2).
- Đợi gợi ý 2-3 thường an toàn hơn.
- **Vấn đề "đoạn mở đầu trùng lặp" của tin tức:** dataset news có intro chuẩn (logo đài, MC dẫn). Các intro này có ngữ nghĩa gần giống nhau giữa nhiều video $\Rightarrow$ vector search dễ trả intro thay vì nội dung chính. **Giải pháp đã ghi trong tài liệu gốc của nhóm là đúng**, mở rộng thêm:
 - **Pre-filter intro/outro offline:** detect intro bằng cách clustering frame trong các keyframe đầu/cuối của nhiều video; cluster có DBSCAN dense + xuất hiện ở $\geq K$ video thì gán nhãn `is_intro`. Mặc định không trả về frame `is_intro=True` trừ khi query match cao với phần nội dung đặc trưng của intro đó.
 - **Re-ranking theo "phân tán cụm":** đúng như ý tưởng nhóm đã có. Có thể formalize bằng metric **Davies-Bouldin Index** hoặc đơn giản hơn là **variance của timestamp** của các top-K frame thuộc cùng video. Variance thấp + đông frame $\Rightarrow$ điểm cao. Variance cao + chia 2 cụm xa nhau $\Rightarrow$ UI bật cảnh báo "ambiguous video, manual check needed".
- **Hậu xử lý gộp video:**

```
Cho top-100 frame từ vector search.
Group by video_id  $\rightarrow$  mỗi nhóm có {frames, scores, timestamps}.
Tính cho mỗi video:
    s_video =  $\alpha \cdot \max(\text{scores}) + \beta \cdot \text{mean}(\text{top-5 scores})$ 
              +  $\gamma \cdot \text{count}(\text{frames in top-100}) \cdot \exp(-\text{variance}(\text{timestamps}))$ 
Sort theo s_video, hiển thị từng video kèm thumbnail cluster.
```

Hyperparam `( $\alpha$ ,  $\beta$ ,  $\gamma$ )` tinh chỉnh trên dev set của các năm trước nếu có.

- **Đa truy vấn từ một câu hỏi:** Mỗi 30s nhóm có thêm gợi ý $\Rightarrow$ thay vì replace query, **append và re-rank**. Có thể cộng dồn các vector embedding bằng mean hoặc weighted average (gợi ý mới quan trọng hơn).

Kỹ thuật bổ sung:

- **Query augmentation bằng LLM:** trước khi search, dùng LLM viết lại câu query Việt thành 3-5 phiên bản (tiếng Việt giàu ngữ nghĩa, dịch tiếng Anh, gạch đầu dòng các entity). Search song song và RRF.
- **Negation handling:** câu có "không có..." rất khó với CLIP/PE (vốn không hiểu phủ định tốt). Solution: parse phủ định bằng LLM, search positive concept, filter out top-K nào match concept đó.

3.2. QA

Bản chất: Full query ngay từ đầu + một câu hỏi cụ thể. Output `<video, frame, answer>`.

Phân loại sâu các loại câu hỏi QA (quan trọng để thiết kế modal đúng):

Loại	Ví dụ	Modal cần thiết
Đếm vật thể trong khung hình	"Bao nhiêu người trong đám cháy?"	Object detection + VLM verify
Nhận dạng thuộc tính	"Túi xách hãng gì?"	OCR + VLM fine-grained
Nội dung tường thuật	"Người dẫn chương trình nói tên ai?"	ASR + LLM
Văn bản trên màn hình	"Tiêu đề bản tin là gì?"	OCR mạnh (PaddleOCR, MMOCR)
Suy luận thời gian/nhân quả	"Sau vụ cháy điều gì xảy ra?"	VLM trên đoạn frame + ASR
Số liệu/thống kê	"Có bao nhiêu nạn nhân?"	ASR + OCR (graphics trên bản tin) + LLM

Đề xuất pipeline QA:

- Stage 1 - Locate:** dùng vector search tìm `<video, frame>` candidates (top-20).
- Stage 2 - Answer:** với mỗi candidate, dùng VLM (Qwen2.5-VL hoặc InternVL3 [cần verify version]) chạy CoT trên frame (hoặc đoạn 3-5 frame liên kề + đoạn ASR ±10s xung quanh) để trả lời.
- Stage 3 - Rerank candidate** bằng cách chấm chất lượng CoT:
 - CoT có dẫn chứng cụ thể từ frame/ASR ⇒ điểm cao.
 - CoT chung chung "có thể là, không chắc" ⇒ điểm thấp.
 - LLM-as-judge có thể tự động hóa bước này.
- Stage 4 - Human-in-the-loop:** UI hiển thị top-5 candidate với CoT đầy đủ. Người vận hành chọn ưng ý nhất, edit `answer` nếu cần, submit.

Kinh nghiệm quan trọng nhóm đã ghi (đồng ý): "thêm VLM xuất CoT và người truy vấn chỉ cần duyệt qua CoT để chọn" - đây là **insight cực kỳ giá trị**. Mở rộng:

- Pre-compute CoT cho top-20 candidate **song song** ngay khi vector search trả về, không đợi người vận hành click. Khi UI render top-K, CoT đã sẵn sàng.
- Latency budget: VLM ~2-3s/frame trên A100 ⇒ 20 frame ≈ 40-60s nếu serial. Cần **batch + parallel** trên nhiều A100. Với 5 account, có thể chạy 5 VLM endpoint song song ⇒ chỉ ~10s.

Câu QA về âm thanh là điểm dễ tách:

- Pre-compute Whisper large-v3 transcript cho **toàn bộ video offline**, lưu kèm timestamp.
- Index transcript trong elasticsearch/Lucene-based + dense (sentence-transformer cho tiếng Việt như `bkai-foundation-models/vietnamese-bi-encoder`) hoặc multilingual E5

[cần verify]).

- Khi query, search song song trên transcript-index, hợp nhất kết quả.

3.3. Visual KIS (V-KIS)

Bản chất: Cho xem video clip 15-20s (không quay phim, không âm thanh, lặp). Tìm `<video, frame>` gốc.

Phân tích vì sao đây là dạng dễ tách nhất:

- Người vận hành phải **địch ngược hình ảnh** → **văn bản** trong đầu, dễ thiếu/sai chi tiết.
- Tốc độ chạy nhanh ⇒ không kịp ghi nhớ đặc tả.
- Không có chữ/âm để bám.
- Vector search text→image không dùng được trực tiếp (vì input là video, không phải text).

Hai hướng giải đã nêu của nhóm - đánh giá:

Solution 1 (Structured caption VLM): Mạnh mẽ, recommended primary.

Thay vì để VLM tự caption thoải mái, **bắt buộc theo template cấu trúc** để mọi keyframe đều có cùng các trường:

```
SCENE_TYPE: [indoor/outdoor/studio/street/...]
CAMERA_ANGLE: [eye-level/high-angle/low-angle/aerial/close-up/wide-shot]
WEATHER: [sunny/cloudy/rainy/night/N/A]
LIGHTING: [bright/dim/backlit/neon/...]
LOCATION_HINTS: [urban/rural/coastal/forest/highway/...]
NUM_PEOPLE: <int>
PEOPLE_DESCRIPTION: [clothing colors, gender/age estimates, actions]
KEY_OBJECTS: [list]
DOMINANT_COLORS: [top 3 hex]
TEXT_ON_SCREEN (OCR): [string]
ACTION_VERBS: [list of action concepts]
NOTABLE_DETAILS: [free-form, 1-2 sentences]
```

Lý do:

- Khi người vận hành xem clip, họ điền vào **cùng cấu trúc** ⇒ matching field-by-field thay vì free-form similarity.
- Tăng recall vì query không bị thiếu ngữ nghĩa (template ép phải nghĩ về đủ chiều).
- Học hỏi từ AIC City Challenge 2026 Track 4 (action description) – cách họ structured caption hành động có thể chuyển sang đây.

Triển khai: Qwen2.5-VL-72B [cần verify] hoặc InternVL3 [cần verify] với prompt template ổn định, batch process toàn bộ keyframe trên 5x A100. Lưu vào ElasticSearch/Postgres dạng nested document để query field-specific.

Solution 2 (GenAI image-from-description + image-search):

- **Điểm yếu chính (đồng ý với nhóm):** GenAI sinh ảnh chậm (5-15s/ảnh), không đủ thực tế trong giải.
- **Image-search năm ngoái không cao** vì các backbone image-similarity (DINOv2, CLIP image-image) tốt cho instance retrieval (cùng vật) nhưng không tốt cho **scene-level similarity** (cùng loại cảnh nhưng khác instance).
- **Cải thiện khả thi:** thay vì sinh ảnh đầy đủ, dùng VLM trích đặc trưng từ clip V-KIS (vì BTC chiếu clip lặp đi lặp lại, có thể quay màn hình không?... không, đề cấm). **Workaround: BTC chiếu trên màn hình trung tâm, nhưng có khi clip được phát qua một protocol/stream mà mình có thể access?** – nếu không, người vận hành phải mô tả bằng tay.

Đề xuất tổ hợp Sol1 + Sol2 thực tế:

1. **Mặc định Sol1:** người vận hành điền template (UI có form-based input có voice support cho từng field). Mỗi field map sang dense + sparse index. Search hybrid.
2. **Background Sol1.5:** cùng lúc, LLM tự generate 5 query phrasings dài hơn từ template; tìm song song.
3. **Fallback Sol2 chỉ khi cần** (Sol1 không trả về kết quả tin cậy sau X giây): GenAI sinh ảnh tham khảo, người vận hành xác nhận → search image-image bằng DINOv2/PE-image-image embedding.

Kỹ thuật bổ sung quan trọng cho V-KIS:

- **Video-level encoder cho indexing:** dùng InternVideo2 hoặc VideoMAE V2 [cần verify] để encode mỗi keyframe + 2-3 frame lân cận thành "clip embedding". Khi V-KIS, dù không quay được clip BTC, vẫn dùng video-aware encoder để index dày hơn về mặt temporal context. Recall@K cao hơn frame-level đơn lẻ ~3-5% trên benchmark MSR-VTT, ActivityNet [cần verify số liệu].
- **Action recognition tag:** chạy model action recognition pretrained (Kinetics-700) gán top-3 action label per shot. Khi clip V-KIS có hành động rõ (cãi nhau, chạy, vẩy tay...), filter cứng theo action label rút ngắn search space cực mạnh.

3.4. TRAKE

Bản chất: Cho query mô tả chuỗi $(E_1, E_2, \dots, E_n)$ ($n=2-4$) theo thời gian trong **một** video. Output $\langle \text{video}, \text{frame1}, \dots, \text{frameN} \rangle$. Epsilon $\pm 10-15$ frame quanh ground truth.

Đây là dạng khó nhất vì:

- Phải định danh đúng video (như T-KIS).
- Phải định danh đúng N điểm thời gian (giống temporal grounding/action localization).
- Sai 1 frame trong N có thể đẩy về 50% accuracy hoặc 0%.

Phân tích bài toán:

TRAKE thực chất là **Video Moment Retrieval (VMR) + Temporal Action Localization (TAL)** đa sự kiện trên cùng video. Hai bước tách rời:

1. **Bước 1 - Locate video:** dùng tổ hợp queries $(E_1 \wedge E_2 \wedge \dots \wedge E_n)$ (vector search hợp nhất), tìm video chứa đầy đủ các sự kiện. Heuristic: video nào có ≥ 1 **frame top-K cho mỗi (E_i)** và **các frame thuộc cùng video xếp theo thứ tự thời gian đúng** $\Rightarrow$ ứng viên cao.
2. **Bước 2 - Locate frames trong video:** với mỗi sự kiện (E_i) , search trong phạm vi video đã định, tìm frame cao điểm nhất. Ràng buộc thứ tự: $\text{timestamp}(\text{frame}_i) < \text{timestamp}(\text{frame}_{i+1})$.

Solution chi tiết:

Algorithm (đặt tên: SequentialEventLocalizer):

```
Input: queries  $[E_1, \dots, E_n]$ , video candidates  $V_{\text{top}}$ 
For each video  $v$  in  $V_{\text{top}}$ :
  For each  $E_i$ :
    Compute  $\text{score}_i(t) = \text{cosine}(\text{embed}(E_i), \text{embed}(\text{frame}_t))$  for all  $t$  in  $v$ 
    Use dynamic programming to find sequence  $(t_1 < t_2 < \dots < t_n)$ 
    maximizing  $\text{sum}(\text{score}_i(t_i))$  subject to  $t_i < t_{i+1}$ 
    + (optional) penalty if gaps too small/large.
Sort video by max sequence score, return top.
```

DP này tương đương bài toán **Longest Increasing Subsequence weighted** - $O(n \cdot T)$ mỗi video, hoàn toàn khả thi với T = số frame trong 1 video ($\sim 1000-3000$).

Trường hợp đặc biệt: TRAKE $n=2-4 \Rightarrow$ DP rất nhẹ, có thể thử ALL ordered combinations.

Tăng cường với caption VLM:

- Captions của frame trong video có thể được nối thành "video narrative".
- Dùng LLM input narrative + queries $[E_1, \dots, E_n]$, output: ước lượng frame range cho mỗi (E_i) (LLM grounding qua frame index).
- Cross-check với DP trên embedding score.

Tăng cường với Audio:

- Whisper transcript có timestamp đến mili-giây. Nếu (E_i) có cue ngôn ngữ ("xe đâm", "tiếng còi"), match transcript snippet $\rightarrow$ narrow window cho frame search.

Fallback 50%: Như đã đề cập, nếu hết 70% thời gian mà còn thiếu sự kiện $\Rightarrow$ submit $(n/2)$ chắc chắn + cấp tốc đoán bù 2 frame còn lại không lý tưởng. Trên kỳ vọng, đoán bù có recall thấp, nhưng có thêm penalty không lớn (vẫn ăn 50%).

Insight rất quan trọng cho UI TRAKE:

- Hiện thị **timeline của video** ở cuối UI khi chọn 1 video candidate. Trên timeline, đánh dấu top-5 frame cho mỗi (E_i) bằng màu khác nhau. Người vận hành kéo-thả 4 frame chốt lên 4

slot kết quả.

- Cho phép **preview clip $\pm 2s$** quanh mỗi frame chọn để xác nhận trước khi submit.
-

4. Phân tích pipeline năm ngoái của nhóm - Bài học

Nhóm đã ghi: "Vector Search (Meta/Perception Encoder) + Metadata search (OCR, Sound...). Metadata search rất tệ và Vector Search chủ yếu mạnh hơn."

Phân tích vì sao metadata search "tệ" năm ngoái:

Khả năng 1: **Trọng số hợp nhất không đúng.** Nếu fusion là $\text{score} = \alpha \cdot \text{vec} + \beta \cdot \text{meta}$ với $\alpha \gg \beta$, metadata không bao giờ thắng vector. Nhưng nếu $\alpha \approx \beta$, các câu vector mạnh sẽ bị metadata nhiều kéo lệch.

Khả năng 2: **Metadata thiếu/sai cho phần lớn video.** YouTube metadata không đồng đều, OCR năm ngoái có thể chất lượng thấp với tiếng Việt có dấu.

Khả năng 3: **Người vận hành không quen workflow hybrid.** Khi vector trả top-100, họ rerank trong đầu thay vì để metadata rerank giúp.

Khuyến nghị 2026:

- **Không bỏ metadata search**, nhưng kích hoạt có điều kiện:
 - OCR-aware search: chỉ kích hoạt khi query chứa từ khóa có khả năng xuất hiện trên màn hình (số, tên riêng, brand name, dấu hiệu trích dẫn). LLM classify query này có "OCR-likely" không.
 - ASR-aware search: chỉ kích hoạt khi query mô tả nội dung lời nói (động từ "nói", "phát biểu", danh xưng).
 - **Trọng số adaptive:** không hardcode (α, β) . Học từ training/dev queries hoặc dùng RRF (Reciprocal Rank Fusion) - không cần tune trọng số, robust tự nhiên.
 - RRF: $\text{score}(d) = \sum_i 1 / (k + \text{rank}_i(d))$, với $k = 60$ thường tốt.
 - **Perception Encoder vẫn là tier-1 backbone**, đồng ý. Bổ sung SigLIP 2 [cần verify version mới nhất] và EVA-CLIP làm ensemble (avg cosine của 3 backbone).
-

5. Brainstorm Solution toàn diện

5.1. Tầng feature extraction (offline)

Mục tiêu: Mọi keyframe đều có ≥ 8 kênh đặc trưng để khi query, không kênh nào "miss".

Kênh	Model gợi ý	Lưu trữ	Ghi chú
Visual dense embedding (primary)	Perception Encoder (Meta) <u>[cần verify variant]</u>	FAISS HNSW	Backbone chính, dim ~1024
Visual dense embedding (secondary)	SigLIP 2 <u>[cần verify]</u> hoặc EVA-CLIP-L	FAISS HNSW	Ensemble
Frame caption (free)	Qwen2.5-VL hoặc InternVL3 <u>[cần verify]</u>	Postgres TEXT + BM25	2-3 câu free-form
Frame caption (structured)	Cùng VLM, template từ §3.3	Postgres JSONB	Field-specific search
OCR (Vietnamese-aware)	PaddleOCR multilingual + VietOCR fallback	Postgres TEXT + BM25	Cần benchmark độ chính xác trên dấu tiếng Việt
ASR	Whisper large-v3	Postgres TEXT + dense (multilingual-E5)	Timestamp granularity 1s
Object detection (open-vocab)	YOLO-World / OWLv2	JSONB (label, bbox, conf)	Bổ sung Faster R-CNN BTC
Scene/Action tag	VideoMAE V2 (Kinetics-700) + Places365	Postgres array + faceted search	Filter cứng
Perceptual hash (dedup)	pHash (16-bit)	Postgres	Loại keyframe trùng giữa BTC và self-cut
Video-clip embedding	InternVideo2 / ViCLIP	FAISS	Cho V-KIS và TRAKE

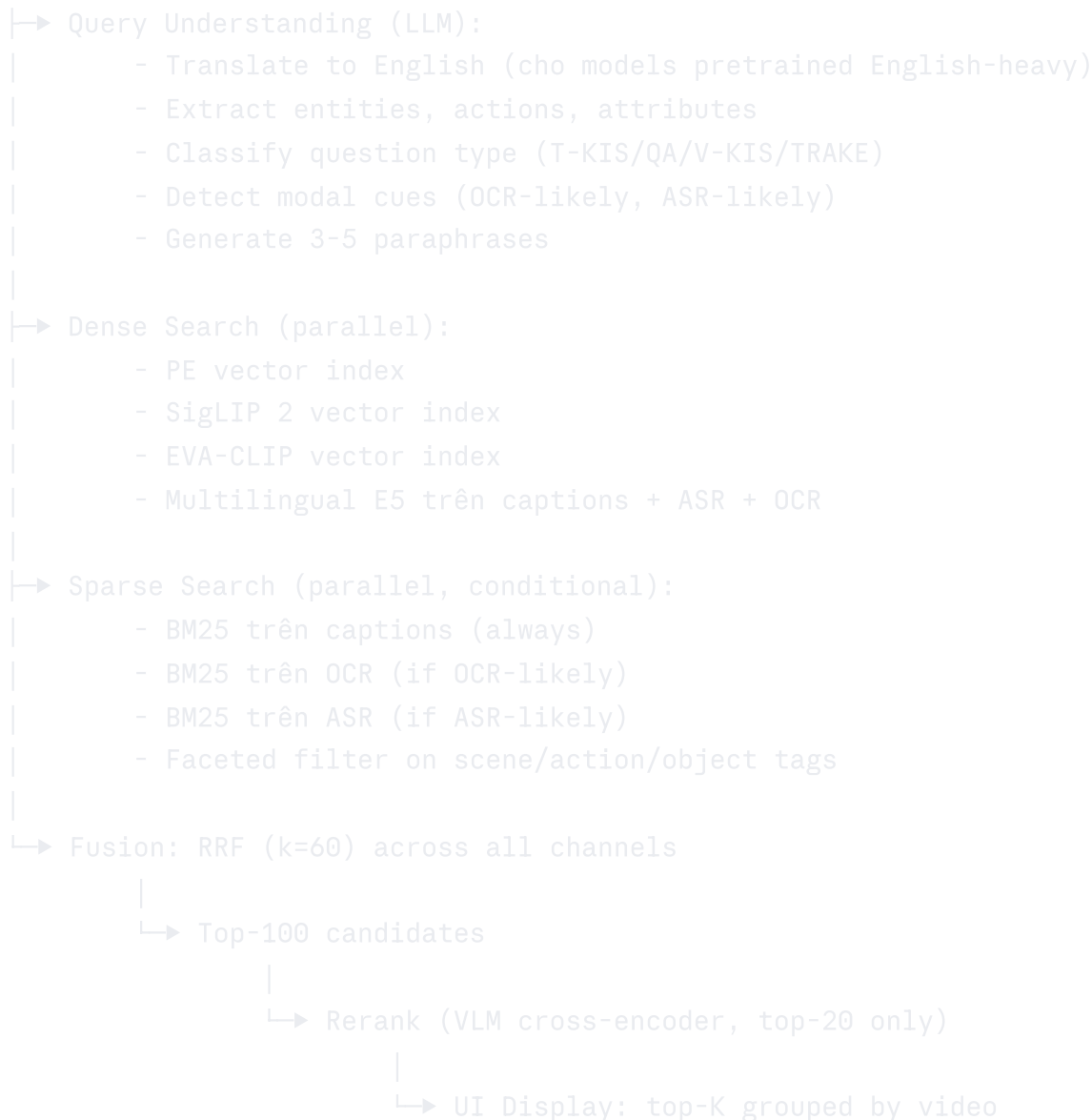
Lưu ý quan trọng:

- **Định danh thống nhất** keyframe_id = (video_id, frame_idx_BTC). Mọi index trỏ về đây.
- Frame self-cut (extra) gán keyframe_id = (video_id, "extra_<i>") và **không submit trực tiếp**; khi rerank cuối, map về frame BTC gần nhất qua timestamp.

5.2. Tầng query & retrieval (online)

Sơ đồ logic:

Query (Vietnamese)



5.3. Tầng rerank & fusion

Hai tầng rerank:

Tầng 1 - RRF/Weighted fusion (không tốn GPU, latency < 10ms):

- Đầu vào: K = 6-8 ranked list từ các channel.
- RRF mặc định, có thể weighted nếu một channel kém chất lượng (vd metadata sparse).

Tầng 2 - VLM cross-encoder rerank (top-20 → top-K final, latency 5-15s trên A100):

- Input: query + image. Output: scalar score.
- Có thể dùng VLM với prompt "Does this image match the description? Answer Yes/No/Partial with confidence 0-1."
- Hoặc dùng cross-modal reranker dedicated như **MM-Cross-Encoder** (nếu có) [cần verify model name].

Tầng 3 (tùy chọn cho QA/TRAKE) - CoT-based rerank:

- VLM xuất CoT giải thích tại sao frame này match. LLM-as-judge chấm CoT.

5.4. Tầng UI/UX

(Chi tiết sâu hơn trong file `AIC26_Pipeline.md`.)

Nguyên tắc thiết kế UI:

1. **Single-screen mode:** mọi thông tin (query, results, candidate detail, submit) cùng 1 màn hình. Không click sang trang khác.
2. **Keyboard-first:** `↔` chọn frame, `Space` preview, `Enter` submit, `Tab` chuyển video, `Esc` hủy.
3. **Voice input:** Whisper local cho voice-to-text, paste vào ô query, LLM rewrite.
4. **Group-by-video display:** mặc định gom frame cùng video, header video kèm metadata (channel, title), expand để xem frame.
5. **Submit guard:** trước khi submit, modal hiển thị `<video, frame>` được chọn + caption + OCR + ASR snippet. Người dùng đọc kỹ → confirm.
6. **Submit history:** sidebar hiển thị các submit đã làm + DRES response (đúng/sai). Tránh resubmit cùng frame.
7. **Multi-tab parallel cho QA/TRAKE:** tab "Locate" + tab "Detail" cho VLM CoT view.
8. **Color-coded timeline cho TRAKE:** timeline ngang dưới video preview, mỗi sự kiện một màu, drag-and-drop để chỉnh.

6. Phân tích lựa chọn model

6.1. Vector backbone chính

Ưu tiên 1: Perception Encoder (Meta)

- Nhóm đã dùng năm ngoái và thấy mạnh. Tiếp tục.
- Variant PE-G hoặc PE-L tùy VRAM. `[cần verify variant chính thức 2026]`

Ưu tiên 2 (ensemble): SigLIP 2 hoặc EVA-CLIP-L

- SigLIP 2 (Google) có sigmoid loss + multilingual capability, lý tưởng cho query tiếng Việt mà không cần dịch.
- EVA-CLIP-L mạnh trên zero-shot retrieval benchmarks.

Không dùng: CLIP ViT-B-32 BTC – quá yếu, chỉ giữ cho sanity check.

6.2. VLM cho captioning và CoT

Ưu tiên 1: Qwen2.5-VL (Alibaba) `[cần verify variant 7B/72B mới nhất]`

- Mạnh trên tiếng Việt (Qwen có training data đa ngôn ngữ).
- Open-weights, chạy được trên A100 (72B với quantization int4/int8).

Ưu tiên 2: InternVL3 [cần verify]

- Cạnh tranh với Qwen, tốt cho dense captioning.

Cho CoT QA: API Gemini 2.5 Flash hoặc Claude Haiku (nếu có budget API)

- Chất lượng CoT thường vượt open-weight ở task suy luận.

6.3. ASR

Whisper large-v3 (OpenAI) – chuẩn industry, tiếng Việt tốt.

- Pre-process: VAD (Voice Activity Detection) bằng Silero VAD để cắt đoạn không có lời, giảm hallucination.
- Output dạng segments với timestamp, lưu vào DB.

6.4. OCR

PaddleOCR (multilingual) – có support tiếng Việt, fast. **VietOCR** (specialized) – fallback nếu PaddleOCR sai dấu nhiều. **MMOCR** – tốt cho scene text khó.

Benchmark cả 3 trên 1000 keyframe random, chọn theo F1 hoặc CER trên tiếng Việt.

6.5. Detection/Action

Open-vocab detection: YOLO-World hoặc OWLv2 – cho phép detect concept text-prompted.

Action recognition: VideoMAE V2 pretrained Kinetics-700 hoặc InternVideo2 [cần verify].

6.6. Reranker

Không có ground truth pretrain phù hợp KIS/QA bằng tiếng Việt. Dùng VLM zero-shot làm reranker thay vì cross-encoder dedicated.

7. Phân bổ tài nguyên 5x A100 Colab Pro

A100 40GB/80GB tùy account. Giả định 40GB worst-case.

Pha offline (chạy 1 lần, có thể nhiều ngày):

Account	Nhiệm vụ	Thời gian ước tính
A1	Extract PE embedding toàn bộ keyframe	T_pe
A2	Extract SigLIP 2 + EVA-CLIP embedding	T_sig
A3	VLM captioning (Qwen2.5-VL) toàn bộ keyframe	T_cap (lớn nhất)
A4	ASR Whisper large-v3 toàn bộ video	T_asr
A5	OCR + Object detection (YOLO-World) + Action tag	T_aux

5 task chạy song song. Bottleneck thường là VLM captioning. Với ~100K keyframe và Qwen2.5-VL-7B int8 batch=8 trên A100 $\Rightarrow$ ~5-10 keyframe/s $\Rightarrow$ ~3-6 giờ. Với 72B int4 $\Rightarrow$ ~1-2 keyframe/s $\Rightarrow$ 14-28 giờ. **Khuyến nghị 7B-int8** cho first pass, 72B-int4 chỉ cho fine subset nếu cần.

Lưu ý Colab Pro: giới hạn runtime ~24h. Cần checkpoint mỗi 500-1000 keyframe để không mất tiến độ.

Pha online (lúc thi):

Account	Endpoint	Latency target
A1	PE/SigLIP encoding service (cho query)	< 500ms
A2	VLM rerank service (top-20 reranker)	< 5s/batch
A3	VLM CoT service cho QA	< 3s/frame, batch
A4	Fallback / overflow	-
A5	Voice-to-text + LLM query rewrite + LLM-as-judge	< 1s

Vector index (FAISS) chạy trên CPU/RAM, không cần GPU lúc query. Postgres/ES cũng vậy. GPU chỉ phục vụ encoding query và VLM rerank/CoT.

Risk: Colab disconnect lúc thi. Mitigation: backup local hoặc thuê 1 VPS A10/A100 ngắn hạn (RunPod, Vast.ai) cho ngày thi chính thức. Colab dùng cho training/indexing, không nên dependency lúc thi.

8. Rủi ro & phương án dự phòng

Rủi ro	Xác suất	Tác động	Mitigation
Colab disconnect lúc thi	Trung bình	Cao	Backup serving trên VPS/local; export FAISS index ra disk; ensemble local CPU-fallback
VLM rerank chậm hơn dự kiến	Trung bình	Trung bình	Giảm top-K rerank (20 → 10); cache rerank kết quả
Query tiếng Việt không có dấu hoặc viết tắt	Cao	Trung bình	LLM rewrite + thử nhiều phiên bản query
OCR sai dấu tiếng Việt	Cao	Trung bình	Multi-OCR ensemble + post-process LLM fix dấu
Keyframe BTC quá thừa cho TRAKE	Trung bình	Cao	Re-extract dense keyframe (1fps hoặc shot-boundary)
V-KIS clip có scene rất hiếm gặp	Trung bình	Cao	Sol2 fallback (GenAI hoặc tăng số phiên bản query)
Người vận hành mệt, mất tập trung	Cao	Cao	Submit guard + rotate người vận hành mỗi 30 phút
Mạng chậm lúc thi	Trung bình	Cao	Tắt cả model serving local, không phụ thuộc API ngoài
Đề có dạng "phủ định" hoặc compositional khó	Cao	Trung bình	LLM parser cho query phức tạp + filter post-search

9. Lộ trình triển khai (gợi ý)

Tuần 1-2: Hạ tầng & ingestion

- Setup FAISS, Postgres, ES.
- Pipeline extract keyframe self-cut (TransNetV2) + dedup pHash.
- Build `KeyframeRegistry`.

Tuần 3-5: Feature extraction (offline)

- Chạy song song 5 A100. Đẩy hết ~100K keyframe + video ASR.
- Checkpoint mỗi 1000 sample.

Tuần 6-7: Indexing & retrieval API

- Build FAISS HNSW cho mỗi backbone.
- Build BM25 + dense ES cho text-fields.
- Test hybrid search trên dev queries (tự tạo hoặc tái sử dụng AIC 2025 queries).

Tuần 8-9: VLM rerank & CoT service

- Stand up Qwen2.5-VL serving endpoint.
- Build CoT prompt template cho QA.
- Benchmark latency.

Tuần 10-12: UI

- React/Next.js hoặc Streamlit (cân nhắc tốc độ phát triển vs trải nghiệm).
- Voice input, group-by-video, submit guard, timeline TRAKE.
- Submit history + DRES integration.

Tuần 13-14: Mock thi

- Tự tạo bộ câu hỏi mock 20 câu/4 dạng.
- Đo điểm trung bình, thời gian/câu, tỉ lệ sai.
- Iterate trên UI và rerank.

Tuần cuối: Tuning + buffer cho phát sinh

- Final benchmark.
- Backup deployment.
- Onboard người vận hành (luyện ngón nóng).

10. Kết luận và 3 ưu tiên cao nhất

1. **Đầu tư UI ngang ngửa đầu tư model.** Phần lớn điểm thua năm ngoái có thể đến từ workflow chứ không phải retrieval quality.
2. **Captioning structured bằng VLM cho V-KIS** là đòn bẩy lớn nhất vì V-KIS là dạng "dễ tách" với input lệch modal.
3. **Submit guard + RRF fusion** là hai biện pháp đơn giản, chi phí thấp, lợi ích cao – nên triển khai sớm.

Mọi quyết định model cụ thể (version Qwen, SigLIP, PE) cần Quýt verify trên Hugging Face / official source trước khi commit vào pipeline production.